

Forecasting Explained

This file explains what the project forecasts, why forecasting matters, how the LSTM models work, and how to see forecasted results.

What We Forecast

The project forecasts two operational signals:

Forecast target	Unit	Why it matters
<code>solar_kw</code>	kW	Predicts available PV generation.
<code>load_kw</code>	kW	Predicts facility demand.

The forecast output is used by:

- Dashboard forecast chart.
- Dispatch planning.
- Alerts.
- Cost optimization.
- Scenario analysis.
- MATLAB/Simulink export workflow.

Why Solar Forecasting Helps

Solar power is variable. It depends on:

- GHI.
- DNI.
- Diffuse irradiance.
- Time of day.
- Season.
- Temperature.
- Humidity/cloud conditions.

If the EMS expects high solar later, it may avoid unnecessary grid charging. If it expects low solar, it can preserve battery or charge earlier.

Example:

```
Forecast says solar will drop sharply at 16:30.  
EMS avoids discharging battery too early.  
Battery remains available for 18:00 to 22:00 peak tariff.
```

Why Load Forecasting Helps

Load is also time-dependent. It depends on:

- Morning start-up demand.
- Daytime operation.
- Evening demand.
- Weekday/weekend behavior.
- Temperature and humidity.
- Random events/spikes.

If the EMS expects high load during peak tariff, it can prepare the battery before that window.

Example:

```
Forecast says load will peak near 20:00.  
Tariff is also high at 20:00.  
EMS charges or preserves battery before the peak.  
Grid import during peak period is reduced.
```

LSTM Input Features

File:

- `backend/app/ml/lstm.py`

The LSTM uses these features:

- Weather: `ghi`, `dni`, `diffuse_irradiance`, `temperature_c`, `wind_speed_mps`, `humidity_pct`, `precipitation_mm`.
- Operational: `solar_kw`, `load_kw`, `tariff_inr_kwh`, `battery_soc_pct`.
- Cyclic time: `hour_sin`, `hour_cos`, `day_sin`, `day_cos`.
- Lag features: `load_lag_1h`, `load_lag_24h`, `solar_lag_1h`, `solar_lag_24h`.
- Rolling averages: `load_roll_3h`, `load_roll_24h`, `solar_roll_3h`, `solar_roll_24h`.

Why Sequence Input Is Used

The model does not look at only one timestamp. It receives a sequence of 48 timesteps.

```
X[t-47], X[t-46], ..., X[t]
```

It predicts the next target value:

```
y[t+1]
```

Why this is useful:

- Solar follows daily pattern.

- Load follows daily and weekly pattern.
- Weather effects can persist across hours.
- LSTM memory gates can learn temporal dependencies.

LSTM Architecture

Parameter	Value
Model type	PyTorch LSTM
LSTM layers	2
Hidden size	96
Dropout	0.15
Dense head	LayerNorm, Linear, ReLU, Dropout, Linear
Loss	Mean Squared Error
Optimizer	AdamW
Sequence length	48
Training device	CUDA

Current Forecasting Results

Model	MAE	RMSE	Device
Solar	2.0748 kW	3.5749 kW	cuda
Load	3.5385 kW	4.7597 kW	cuda

Interpretation:

- Solar error is low enough to track PV shape.
- Load error is low relative to the observed peak load of 151.913 kW.
- Forecasts support dispatch decisions but should not be treated as perfect truth.

Where Forecast Files Are Saved

Output	Path
Solar model	<code>models/forecast/solar_kw_lstm.pt</code>
Load model	<code>models/forecast/load_kw_lstm.pt</code>
Solar forecast CSV	<code>data/outputs/solar_kw_forecast.csv</code>
Load forecast CSV	<code>data/outputs/load_kw_forecast.csv</code>
Solar training history	<code>data/outputs/solar_kw_training_history.csv</code>
Load training history	<code>data/outputs/load_kw_training_history.csv</code>

Output	Path
Solar training report	data/outputs/solar_kw_training_report.json
Load training report	data/outputs/load_kw_training_report.json

Forecast Plots

Plot	Path	Meaning
Solar prediction vs actual	data/outputs/plots/solar_kw_prediction_vs_actual.png	Visual comparison of predicted and actual solar output.
Load prediction vs actual	data/outputs/plots/load_kw_prediction_vs_actual.png	Visual comparison of predicted and actual load demand.
Solar loss curve	data/outputs/plots/solar_kw_loss_curve.png	Training and validation loss for solar model.
Load loss curve	data/outputs/plots/load_kw_loss_curve.png	Training and validation loss for load model.

How To See Forecasted Results

In The Dashboard

1. Start backend and frontend.
2. Open:

```
http://127.0.0.1:5173
```

3. Look at the **Forecast** panel.

It shows:

- Forecast solar kW.
- Forecast load kW.

Through API

Open:

```
http://127.0.0.1:8000/forecast?horizon_hours=24
```

Example response shape:

```
{
  "horizon_hours": 24,
  "model_status": {
    "source": "lstm_artifacts"
  },
  "records": [
    {
      "timestamp": "...",
      "solar_kw": 42.1,
      "load_kw": 76.4
    }
  ]
}
```

From CSV

Open:

```
data/outputs/solar_kw_forecast.csv
data/outputs/load_kw_forecast.csv
```

Each contains:

- Timestamp.
- Actual value.
- Predicted value.

Forecast Source Priority

File:

- `backend/app/ml/inference.py`

The API chooses forecast source in this order:

1. If live simulator is running, use live simulator forecast.
2. Else if trained LSTM forecast artifacts exist, use LSTM forecast CSVs.
3. Else use persistence fallback based on recent historical patterns.

This prevents the dashboard from breaking even if models are not trained yet.

How Forecasting Helps Dispatch

Forecasting helps answer:

- Will solar be available later?
- Will load rise soon?

- Should battery be preserved?
- Should battery be pre-charged?
- Is there risk of peak demand?
- Is a renewable drop coming?

Example decision:

```
Current time: 15:00  
Forecast: solar falling, load rising, peak tariff starts at 18:00  
Decision: avoid unnecessary discharge now, keep SoC available for evening
```

Defense For Forecasting In Viva

Say:

Forecasting is needed because battery dispatch is a temporal decision. A controller that only sees the present can waste battery energy before a more expensive or critical period. The LSTM gives the EMS a forward-looking estimate of solar and load, allowing it to schedule battery behavior more intelligently.

If asked why LSTM:

LSTM is suitable because both solar and load are sequential time-series signals with daily, weekly, and weather-driven dependencies. The model uses 48-step sequences and engineered lag/rolling features, so it can learn temporal behavior better than a single-row regressor.

If asked about limitations:

Forecast uncertainty increases with horizon length. The project therefore uses forecasts for near-term operational planning and keeps battery safety constraints hard-coded so forecast errors cannot violate SoC limits.